

Integración de Servicios Cloud GCP

Servicios, sockets, servidores, DNS y permisos en Linux

Semana 04 - Sesión 07

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Ya instalaron Visual Studio Code, Warp y Antigravity CLI?
- ▶ ¿Qué comando usaron para encontrar la puerta de enlace de la VM?
- ▶ ¿Qué protocolo y puerto encuentran al abrir `http.rip` en el navegador?

Logro de la sesión

Al finalizar, podrás explicar cómo un servicio atiende clientes, escribir servidores y clientes simulados en Python, usar servidores y servicios reales (Apache, MySQL y el firewall del sistema), reconocer los puertos clásicos, describir el papel de un servidor DNS y administrar los permisos de los archivos de un proyecto.

- ▶ ¿Se acuerdan de las capas del modelo OSI?
- ▶ ¿Ya crearon servidores en Node.js o Python?

¿Para qué te servirá lo aprendido?

Casi todo servicio cloud que uses —una API, una base de datos, un balanceador de carga— es un programa escuchando en un socket. Escribir uno pequeño te permite entender qué ocurre cuando algo no responde.

También podrás abrir y cerrar puertos con criterio, entender cómo el DNS apunta un dominio a un servidor y proteger archivos sensibles como el `.env` de un proyecto.

Servicios, sockets y modelo cliente-servidor

Un servicio es un programa que espera peticiones

Idea central

El **cliente** inicia la conversación y pide algo. El **servidor** no hace nada hasta que alguien le pide: permanece a la espera, atiende y vuelve a esperar.



Servicio o *daemon*

Un programa que el sistema mantiene corriendo en segundo plano, sin ventana ni terminal propia, administrado por `systemd`.

Un mismo equipo hace ambos papeles

Tu VM es servidor cuando publica una API y cliente cuando consulta `google.com`.

El socket es el punto donde se atiende la conexión

Qué lo define

Un socket queda identificado por tres datos: **protocolo** (TCP o UDP), **dirección IP** y **puerto**. Por eso una misma IP puede atender muchos servicios a la vez.

Los cuatro pasos del servidor

```
socket() crea el punto de conexión  
bind() lo fija a IP y puerto  
listen() abre la cola de espera  
accept() atiende a un cliente
```

La dirección de escucha importa

127.0.0.1 solo acepta clientes de la propia máquina.

Un puerto, un servicio

Si dos programas piden el mismo puerto, el segundo falla con Address already in use.

Por qué se puede escribir a mano

HTTP no tiene nada de mágico: el cliente envía líneas de texto y el servidor devuelve otras líneas de texto. Lo único obligatorio es respetar el formato: una línea en blanco separa las cabeceras del contenido.

Lo que envía el cliente

```
GET / HTTP/1.1  
Host: 127.0.0.1:8080  
(línea en blanco)
```

Método, ruta y versión; luego las cabeceras.

Lo que responde el servidor

```
HTTP/1.1 200 OK  
Content-Type: text/plain  
Content-Length: 41  
(línea en blanco)  
Hola desde un servidor...
```

Código de estado, cabeceras y contenido.

Núcleo del script

```
import socket
s = socket.socket(socket.AF_INET,
    socket.SOCK_STREAM)
s.bind(('127.0.0.1', 8080))
s.listen(5)
while True:
    cli, dir = s.accept()
    cli.recv(1024)
    cli.sendall(Respuesta.encode())
    cli.close()
```

Pruébalo

```
$ python3 servidor_http.py
Escuchando en
http://127.0.0.1:8080

$ curl -i 127.0.0.1:8080
HTTP/1.1 200 OK
Hola desde un servidor
hecho con sockets
```

1. Con el servidor corriendo, pregunta a agy en otra terminal

Tengo servidor_http.py escuchando en el puerto 8080. Dame los comandos de Ubuntu para ver qué puertos están en escucha y qué proceso abrió cada uno, averiguar el PID que ocupa el 8080, y mostrar el nombre y las direcciones IP de esta máquina. Explica qué significa cada columna de la salida.

2. Anota lo que encuentres

```
PUERTO    = 8080
PID       = [? ]
PROCESO   = [? ]
DIRECCION = [? ]
HOSTNAME  = [? ]
```

Junto a cada dato, anota el comando que usaste y pide a agy que lo explique.

3. Detén el proceso por su PID

```
$ kill [PID ]
$ curl 127.0.0.1:8080
curl: (7) Failed to connect
```

Si no termina, kill -9 lo fuerza. Busca de nuevo el puerto: ya no debe aparecer.

Qué debes poder explicar

Por qué el 8080 solo acepta conexiones de la propia máquina, y qué cambia al buscar el puerto antes y después del kill.

Núcleo del script

```
import socket
c = socket.socket(socket.AF_INET,
    socket.SOCK_STREAM)
c.connect(('127.0.0.1', 8080))
c.sendall(PETICION.encode())
respuesta = c.recv(1024)
c.close()
print(respuesta.decode())
```

Es lo mismo que hace curl, escrito a mano.

El cliente hace menos pasos

```
socket() crea el punto
connect() busca al servidor
sendall() envía la petición
recv() lee la respuesta
```

No hay bind() ni listen(): el cliente no espera a nadie.

El mismo esqueleto, otra respuesta

```
TABLA = [(1, 'Ana', 'ana@utp.edu.pe'), ... ]

s.bind(('127.0.0.1', 3307))
s.listen(5)
while True:
    cli, dir = s.accept()
    consulta = cli.recv(1024).decode()
    cli.sendall(responder(consulta).encode())
    cli.close()
```

Qué cambia frente al de HTTP

Solo el puerto y lo que se devuelve: en vez de una página, filas de una tabla.

Puerto 3307

Se usa 3307 para no chocar con un MySQL real, que ocupa el 3306.

Conecta, consulta y lee

```
c.connect(('127.0.0.1', 3307))
c.sendall('SELECT * FROM usuarios'.encode())
respuesta = c.recv(4096)
c.close()
print(respuesta.decode())
```

Resultado

```
$ python3 cliente_mysql.py
Consulta enviada:
SELECT * FROM usuarios
1 | Ana | ana@utp.edu.pe
2 | Luis | luis@utp.edu.pe
3 | Sara | sara@utp.edu.pe
```

La misma secuencia que el cliente HTTP

`connect()`, `sendall()`, `recv()`. Cambia el puerto y el texto que se envía; el mecanismo es idéntico. Un MySQL real usa un formato binario más elaborado, pero por debajo también es esto.

Cliente

Quien inicia la conexión y pide algo. Termina cuando obtiene su respuesta.

Servidor

Quien escucha en un puerto y responde. Necesita estar activo antes que el cliente.

Servicio

Cualquier programa que el sistema mantiene en segundo plano para cumplir una función.

No todo servicio es un servidor

Un servidor siempre escucha peticiones; un servicio no tiene por qué hacerlo. `cron` ejecuta tareas a cierta hora, un servicio de respaldo copia archivos de madrugada y `systemd-timesyncd` ajusta el reloj: ninguno abre un puerto ni espera clientes, y aun así los tres son servicios.

El firewall es un servicio, no un servidor

Nadie se conecta al firewall. Se ejecuta en segundo plano revisando el tráfico que entra y decidiendo si dejarlo pasar. Por eso no tiene cliente ni responde peticiones: solo filtra.

Cómo razona

Recorre una lista de **reglas** en orden. Si alguna coincide con el origen y el puerto, aplica su acción. Si ninguna coincide, aplica la **política por defecto**.

Política por defecto: denegar

Se cierra todo y se abre solo lo necesario. Es el criterio recomendado en servidores.

Qué mira cada regla

Dirección de origen, puerto de destino, protocolo y la acción: permitir o denegar.

Escuchar no es ser accesible

El programa puede estar activo y aun así el firewall impedir que lleguen las conexiones.

Reglas y decisión

```
POLITICA_POR_DEFECTO = 'DENEGAR'
REGLAS = [
    {'origen': '192.168.1.0/24',
     'puerto': 22, 'accion': 'PERMITIR'},
    {'origen': '0.0.0.0/0',
     'puerto': 80, 'accion': 'PERMITIR'},
    ...
]
```

Resultado

```
$ python3 firewall.py
Política por defecto: DENEGAR

192.168.1.45 -> puerto 22 PERMITIR
203.0.113.9 -> puerto 22 DENEGAR
203.0.113.9 -> puerto 80 PERMITIR
203.0.113.9 -> puerto 3306 DENEGAR
127.0.0.1 -> puerto 3306 PERMITIR
```

El mismo origen entra por el 80 y no por el 3306.

Servidores y servicios reales en uso

Instala y comprueba

```
$ sudo apt update
$ sudo apt install apache2 -y
$ systemctl status apache2
$ curl -I 127.0.0.1
HTTP/1.1 200 OK
Server: Apache/2.4.58 (Ubuntu)
```

Qué aporta frente al nuestro

Atiende muchos clientes a la vez, sirve archivos reales, registra accesos y admite HTTPS.

Dónde viven los archivos

```
/var/www/html/index.html
```

Lo que dejes ahí queda publicado.

Instala, activa y conéctate

```
$ sudo apt install mysql-server -y
$ sudo systemctl start mysql
$ sudo systemctl enable mysql
$ systemctl is-active mysql
active
$ sudo mysql -e 'SHOW DATABASES;'
```

Lo que simulamos, en real

Aquí también hay un servidor escuchando y un cliente (`mysql`) que envía consultas y recibe filas.

Comprueba el nombre real

Con MariaDB la unidad cambia de nombre. Búscala con `systemctl list-unit-files '*mysql*'.`

Política por defecto y permisos

```
$ sudo ufw default deny incoming
$ sudo ufw allow 22/tcp
$ sudo ufw allow 80/tcp
$ sudo ufw enable
$ sudo ufw status numbered
```

Abre el 22 antes de activarlo: si administras la máquina por SSH y no lo permites, pierdes el acceso.

Es el mismo razonamiento

default deny es la política por defecto de firewall.py; cada allow es una regla de la lista.

Regla por origen

```
$ sudo ufw allow from \
192.168.1.0/24 to any \
port 3306
```

Cerrar la base de datos al exterior

La app se conecta desde la propia máquina o desde la red interna; el 3306 nunca debe estar abierto a Internet.

Publicar solo lo necesario

En un servidor web se dejan abiertos 80 y 443, y se cierra todo lo demás.

Limitar la administración

Permitir el 22 solo desde la IP de la oficina o de la VPN, no desde cualquier origen.

Bloquear un origen abusivo

Banear la IP que está intentando adivinar contraseñas o saturando el servidor.

Cerrar puertos de desarrollo

3000, 8000 y paneles internos sirven mientras desarrollas; en producción no deben quedar expuestos.

Separar ambientes

Que el servidor de pruebas solo acepte conexiones del equipo de desarrollo.

1. Publica un sitio estático en la VM

```
$ echo '<h1>Sitio de prueba</h1>' \  
| sudo tee /var/www/html/index.html  
$ sudo ufw allow 80/tcp  
$ sudo ufw enable
```

Desde el host, abre `http://VM_IP` en el navegador: debe verse la página.

2. Banea la IP del host

```
$ sudo ufw insert 1 deny from HOST_IP
```

3. Comprueba y revierte

```
$ sudo ufw status numbered  
[1]Anywhere DENY IN HOST_IP  
[2]80/tcp ALLOW IN Anywhere  
  
$ sudo ufw delete 1
```

Recarga el navegador en cada paso y anota qué ocurre.

deny frente a reject

deny descarta el paquete en silencio: el navegador se queda cargando hasta agotar el tiempo. reject avisa y el error es inmediato.

Servicios y puertos clásicos

Por qué importan

El puerto indica a cuál de esos servicios llega cada conexión. Los números que siguen son **estándares**: la convención que todo el mundo espera. Pero, como ya vieron, nada impide levantar un servidor en el puerto que quieran —lo hicimos en el 8080 y en el 3307—; solo que entonces el cliente debe indicarlo.

Acceso y web

- 22 SSH: administración remota.
- 80 HTTP: web sin cifrar.
- 443 HTTPS: web cifrada.
- 53 DNS: resolución de nombres.

Datos y aplicaciones

- 3306 MySQL.
- 5432 PostgreSQL.
- 8000 y 3000: aplicaciones Node o Python en desarrollo.
- 25 y 587 SMTP: correo saliente.

Qué hace un analista de seguridad ante un puerto abierto

Cada puerto abierto es superficie de ataque

El trabajo no es cerrarlo todo, sino justificar uno por uno: qué servicio escucha ahí, quién necesita alcanzarlo y desde dónde.

1. Inventariar

```
$ sudo ss -ltnp
```

Qué escucha, en qué dirección y qué programa lo abrió.

2. Mirar desde fuera

```
$ nmap -sV VM_IP
```

Ve lo mismo que vería un atacante, incluida la versión del servicio.

3. Preguntar por cada puerto

¿Debe ser accesible desde Internet, solo desde la red interna o solo desde la propia máquina?

4. Reducir y vigilar

Cerrar lo que no se justifica, restringir por origen, mantener el servicio actualizado y revisar los registros de acceso.

El servidor DNS

Qué problema resuelve

Las personas recordamos `google.com`; las máquinas necesitan una IP. El DNS es el servicio que convierte lo primero en lo segundo, y escucha en el puerto 53. Si falla, el navegador no encuentra el sitio aunque el servidor esté funcionando.

Registros frecuentes

A: apunta un nombre a una IPv4.

AAAA: lo mismo, con IPv6.

CNAME: alias de otro nombre.

MX: a qué servidor va el correo.

TXT: notas y verificaciones de dominio.

Lo que harás en la práctica

Al contratar un hosting o crear una VM te entregan una **IP pública**. Para que tu dominio apunte ahí, entras al panel del proveedor del dominio y editas el **registro A**, poniendo esa IP.

Y el `www`

Suele resolverse con un CNAME de `www.tudominio.com` hacia `tudominio.com`, para no repetir la IP en dos sitios.

Gestión de permisos

Idea central

Ubuntu protege cada archivo indicando qué acciones puede realizar el **propietario**, el **grupo** y el **resto de usuarios**. Un servidor lo usan varias personas y varios servicios: los permisos evitan que un programa cualquiera lea las credenciales de otro.

¿Quién?

u: propietario.

g: grupo.

o: otros usuarios.

¿Qué puede hacer?

r: leer.

w: modificar.

x: ejecutar o atravesar un directorio.

Los archivos del proyecto

```
rgap@ubuntu:~/proyecto$ ls -l
-rw-rw-r-- 1 rgap rgap 223 Sep 1 13:51 .env
-rw-rw-r-- 1 rgap rgap 1339 Sep 1 13:50 servidor_http.py
```

-	rw-	rw-	r--
Archivo regular. Una d indicaría directorio.	El propietario lee y modifica; todavía no ejecuta.	El grupo rgap lee y modifica.	Los demás solo leen. Aquí está el problema del .env.

Caso 1: un .py necesita intérprete y permiso x

Primera línea del script

```
#!/usr/bin/env python3
import socket
...
```

Le indica a Ubuntu qué programa debe interpretarlo.

Por qué python3 archivo.py sí funciona

Ahí ejecutas Python, y Python solo lee el archivo.
./archivo.py exige que el archivo mismo sea ejecutable.

Antes y después

```
$ ./servidor_http.py
bash: Permiso denegado
$ chmod u+x servidor_http.py
$ ./servidor_http.py
Escuchando en
http://127.0.0.1:8080
```


Caso 2: el .env guarda secretos y debe cerrarse

Qué contiene

```
DB_USER=app_web
DB_PASSWORD=cambia-esta-clave
API_KEY=clave-de-ejemplo-no-real
```

Credenciales que dan acceso real a la base de datos y a servicios de pago.

Ciérralo al propietario

```
$ chmod 600 .env
$ ls -l .env
-rw----- 1 rgap rgap 223 .env
```

600: el propietario lee y escribe; nadie más entra.

El permiso no reemplaza al .gitignore

chmod protege el archivo en esta máquina. Para que no llegue al repositorio, además debe estar listado en .gitignore.

Simbólica: dices qué cambiar

```
$ chmod u+x app.py  
$ chmod o-r .env
```

La primera agrega ejecución al propietario. La segunda quita la lectura a los demás.

Los tres que verás casi siempre

```
chmod 600 .env rw- --- --- solo el propietario  
chmod 644 index.html rw- r-- r-- todos leen, solo tú editas  
chmod 777 archivo rwx rwx rwx todos hacen todo: evítalo
```

777 no arregla la causa de un error de permisos; solo amplía quién puede provocarlo.

Numérica: $r=4$, $w=2$, $x=1$

```
4+2+1 = 7  rwx  
4+2 = 6  rw-  
4 = 4  r--
```

Un dígito por cada grupo: propietario, grupo y otros.

Práctica: sitio en React generado con IA y sus permisos

(20 min)

1. Pide el proyecto a agy o a Warp

Crea un sitio estático muy simple en React con Vite que muestre una lista de usuarios traída de <https://jsonplaceholder.typicode.com/users>. Una sola pantalla, sin router ni librerías extra. Agrega un `.env` con `VITE_API_URL`, un script `iniciar.sh` que ejecute `npm install` y `npm run dev`, y un `.gitignore` que excluya `.env` y `node_modules`. Antes de escribir nada, explica el plan.

2. Revisa lo que quedó

```
$ ls -la
$ cat .gitignore
$ ./iniciar.sh
bash: Permiso denegado
```

3. Corrige los permisos

```
$ chmod u+x iniciar.sh
$ chmod 600 .env
$ ls -l iniciar.sh .env
```

Comprobación y entrega

`iniciar.sh` debe empezar con `-rwx` y `.env` con `-rw-----`. El sitio debe abrir y mostrar los usuarios. Explica en una línea por qué cada archivo lleva ese permiso y no otro.



gracias



Universidad
Tecnológica
del Perú

- ▶ Documentación de Python: módulo `socket` y guía `Socket Programming HOWTO`, docs.python.org/3/library/socket.html.
- ▶ MDN Web Docs: *An overview of HTTP*, developer.mozilla.org/docs/Web/HTTP.
- ▶ Documentación de Ubuntu Server: Apache, MySQL, UFW y `systemd-resolved`, documentation.ubuntu.com/server.
- ▶ Páginas de manual de `ufw`, `ss` y `nmap`.
- ▶ `systemd`: páginas de manual de `systemctl`, `systemd.unit` y `systemd.service`, freedesktop.org/software/systemd/man.
- ▶ RFC 1034 y RFC 1035: conceptos y especificación del sistema de nombres de dominio.
- ▶ GNU Coreutils: manual de `chmod`; páginas de manual de `ls`, `chmod` y `env`.
- ▶ Documentación oficial de Vite y de React, vite.dev y react.dev.
- ▶ Documentación oficial de Google Antigravity CLI, antigravity.google/docs/cli.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*; identidad visual y estructura docente de la Sesión 05.